

Quiero reestructurar la arquitectura de mi aplicación Ladder/PLC para alinearla con el enfoque definido por el profesor y con la forma real en que vamos a trabajar el proyecto.

Contexto general del proyecto: Estamos desarrollando una herramienta web donde el usuario puede pedir en lenguaje natural un comportamiento lógico para un PLC. La aplicación debe interpretar esa intención, convertirla a una representación lógica simple en JSON y mostrarla visualmente en una herramienta de diagramas Ladder. Posteriormente, si el usuario decide ejecutar esa lógica en el PLC físico, ese mismo JSON lógico viajará al backend para que el backend haga la traducción a registros y modifique la configuración del PLC.

Punto clave de arquitectura: NO queremos que la IA genere geometría Ladder. NO queremos que la IA genere filas, columnas, ramas, spans ni posiciones visuales. NO queremos que la IA genere directamente tabla de verdad completa. NO queremos que la IA escriba registros PLC directamente. NO queremos que el frontend dependa de que la IA "dibuje bien".

La IA debe hacer únicamente esto: Interpretar la instrucción de alto nivel del usuario y devolver un JSON lógico simple, estructurado, validable y fácil de transformar.

La parte de tabla de verdad / interpretación lógica de bajo nivel queda resuelta del lado del programa maestro del PLC y/o del traductor backend. En esta etapa del frontend no vamos a generar una tabla de verdad explícita como salida principal. En lugar de eso, la IA debe producir una descripción lógica declarativa, y el frontend debe compilar esa descripción a una vista Ladder entendible para el usuario.

Flujo objetivo de la arquitectura:

Usuario escribe una instrucción: "Quiero que la lámpara verde prenda con el botón verde o con el botón amarillo, pero que se apague con el paro general."

↓

IA interpreta intención

↓

IA devuelve JSON lógico simple:

```
{ "name": "Lámpara verde con dos botones y paro general", "logic_type": "combinational", "outputs":  
[ { "coil": "Q10", "expr": "(I1 + I3) * !I7", "mode": "direct", "comment": "Q10 prende con I1 o I3, siempre  
que no esté activo el paro general" } ], "timers": [], "states": [], "global_rules": { "global_stop": "I7",  
"stop_priority": true } }
```

↓

Frontend valida el JSON

↓

Frontend transforma ese JSON a una visualización Ladder

↓

Usuario revisa el diagrama

↓

Si el usuario decide ejecutar: Ese mismo JSON viaja al backend

↓

Backend traduce el JSON a la estructura que necesite el PLC: registros, marcas, modos, timers, flags, etc.

↓

PLC ya tiene un programa maestro fijo que interpreta esa configuración y activa salidas físicas.

Objetivo inmediato: Dejar impecable el frontend y el traductor de intención de usuario a JSON simple. No enfocarse todavía en la escritura real de registros PLC, porque esa parte la está trabajando otro integrante del equipo. La prioridad es que la app sea robusta para: 1. Recibir instrucciones en lenguaje natural. 2. Pedir a la IA un JSON lógico simple. 3. Validar ese JSON. 4. Mostrarlo en la herramienta visual Ladder. 5. Preparar el JSON para que después pueda enviarse al backend de ejecución.

El JSON simple debe ser la fuente de verdad del frontend.

Estructura recomendada del JSON lógico:

```
{ "name": "string", "logic_type": "combinational | stateful | timed | mixed", "device_profile":  
"maletin_basico", "inputs_used": ["I1", "I2", "I3"], "outputs": [ { "coil": "Q10", "expr": "I1 + I3", "mode":  
"direct", "comment": "string" } ], "timers": [ { "id": "T1", "type": "TON | TOF | OSCILLATOR", "enable": "I1",  
"preset_ms": 5000, "comment": "string" } ], "states": [ { "id": "M1", "type": "latch", "set": "I1", "reset": "I2",  
"comment": "string" } ], "global_rules": { "global_stop": "I7", "stop_priority": true }, "warnings": [] }
```

Reglas del JSON:

- `outputs` es obligatorio.
- Cada salida debe tener `coil`, `expr`, `mode` y `comment`.
- `expr` debe usar una sintaxis booleana simple:
- `*` o `&` para AND
- `+` o `|` para OR
- `!` para NOT
- paréntesis para agrupar
- Ejemplos válidos:
- `Q10 = I1`
- `Q10 = I1 + I3`
- `Q10 = I1 * I4`
- `Q10 = (I1 + I3) * !I7`
- `Q10 = M1`
- `Q10 = T1.DN`
- La IA no debe usar direcciones físicas tipo `%R10`, `%M1`, `%Q10` salvo que el perfil del dispositivo lo requiera.
- Para el frontend, usar nombres lógicos simples: `I1`, `I2`, `I3`, `I4`, `I7`, `Q10`, `Q11`, `Q12`, `M1`, `T1`.
- El mapeo físico a registros PLC se deja para el backend.

Tipos de lógica a soportar:

1. Lógica directa: Usuario: "Con I1 prende Q10."

JSON: { "outputs": [{ "coil": "Q10", "expr": "I1", "mode": "direct", "comment": "Q10 sigue directamente a I1" }] }

1. Lógica OR: Usuario: "Q10 debe prender con I1 o con I3."

JSON: { "outputs": [{ "coil": "Q10", "expr": "I1 + I3", "mode": "direct", "comment": "Q10 prende si I1 o I3 están activos" }] }

1. Lógica AND: Usuario: "Q10 prende solo si I1 está activo y el selector I4 está activo."

JSON: { "outputs": [{ "coil": "Q10", "expr": "I1 * I4", "mode": "direct", "comment": "Q10 prende solo cuando I1 e I4 están activos" }] }

1. Paro general: Usuario: "Q10 prende con I1, pero se apaga si activo el paro general."

JSON: { "outputs": [{ "coil": "Q10", "expr": "I1 * !I7", "mode": "direct", "comment": "Q10 prende con I1 siempre que I7 no esté activo" }], "global_rules": { "global_stop": "I7", "stop_priority": true } }

1. Enclavamiento: Usuario: "Con I1 arranca Q10 y se queda encendida. Con I2 se apaga."

JSON: { "logic_type": "stateful", "outputs": [{ "coil": "Q10", "expr": "M1", "expr_next": "(I1 + M1) * !I2", "mode": "latched", "comment": "Q10 queda enclavada con I1 y se libera con I2" }], "states": [{ "id": "M1", "type": "latch", "set": "I1", "reset": "I2", "comment": "Memoria interna para enclavamiento de Q10" }] }

1. Timer TON: Usuario: "Cuando presione I1, espera 5 segundos y prende Q10."

JSON: { "logic_type": "timed", "outputs": [{ "coil": "Q10", "expr": "T1.DN", "mode": "timer_on_delay", "comment": "Q10 prende cuando el temporizador T1 termina" }], "timers": [{ "id": "T1", "type": "TON", "enable": "I1", "preset_ms": 5000, "comment": "Temporizador de retardo a la conexión activado por I1" }] }

1. Parpadeo: Usuario: "Mientras I1 esté activo, Q12 debe parpadear cada segundo."

JSON: { "logic_type": "timed", "outputs": [{ "coil": "Q12", "expr": "I1 * BLINK_1S", "mode": "blinker", "comment": "Q12 parpadea mientras I1 esté activo" }], "timers": [{ "id": "BLINK_1S", "type": "OSCILLATOR", "enable": "I1", "preset_ms": 1000, "comment": "Oscilador de 1 segundo para parpadeo" }] }

Responsabilidad del frontend: El frontend debe tomar el JSON lógico simple y convertirlo a una representación visual Ladder.

Ejemplo: Si recibe:

{ "outputs": [{ "coil": "Q10", "expr": "I1 + I3", "mode": "direct" }] }

Debe mostrar algo equivalente a:

I1 ———— | ———— (Q10) I3 ————

Si recibe:

{ "outputs": [{ "coil": "Q10", "expr": "I1 * I4", "mode": "direct" }] }

Debe mostrar:

I1 — I4 ——— (Q10)

Si recibe:

```
{ "outputs": [ { "coil": "Q10", "expr": "(I1 + I3) * !I7", "mode": "direct" } ] }
```

Debe mostrar una rama paralela para I1/I3 y después un contacto NC para I7 antes de la bobina Q10.

Responsabilidad del traductor IA: Crear un endpoint o módulo que reciba texto del usuario y devuelva únicamente el JSON lógico simple.

Ejemplo de entrada: { "texto": "Haz que la lámpara verde prenda con el botón verde o el amarillo, pero que se apague con el paro general", "device_profile": "maletin_basico" }

Ejemplo de salida: { "name": "Lámpara verde con dos botones y paro general", "logic_type": "combinational", "device_profile": "maletin_basico", "inputs_used": ["I1", "I3", "I7"], "outputs": [{ "coil": "Q10", "expr": "(I1 + I3) * !I7", "mode": "direct", "comment": "Q10 prende con I1 o I3, siempre que no esté activo I7" }], "timers": [], "states": [], "global_rules": { "global_stop": "I7", "stop_priority": true }, "warnings": [] }

Reglas para el system prompt de la IA:

- Responde exclusivamente con JSON válido.
- No agregues explicación fuera del JSON.
- No generes geometría Ladder.
- No generes tabla de verdad.
- No generes registros PLC.
- No generes código Python, Modbus ni Cscape.
- No inventes entradas o salidas fuera del perfil del dispositivo.
- Si la instrucción es ambigua, llena `warnings` con la duda y usa la interpretación más segura.
- Si el usuario pide una función temporal, usa `timers`.
- Si el usuario pide memoria, mantener, enclavar, quedarse encendido o apagar con otro botón, usa `states`.
- Si el usuario pide paro general, usa `global_rules.global_stop`.
- Si hay conflicto entre arranque y paro, el paro tiene prioridad.
- Para el frontend, toda salida debe terminar representándose como una bobina.
- Para contactos negados, usa `!`.

Validaciones mínimas del frontend:

- Confirmar que `outputs` existe y tiene al menos una salida.
- Confirmar que cada `coil` existe en el perfil del dispositivo.
- Confirmar que cada variable usada en `expr` existe como entrada, salida, estado o timer.
- Confirmar que los paréntesis de `expr` están balanceados.
- Confirmar que no existan símbolos no soportados.
- Confirmar que si una expresión usa `T1.DN`, entonces exista `T1` en `timers`.
- Confirmar que si una expresión usa `M1`, entonces exista `M1` en `states`.
- Si algo no pasa validación, mostrar warning claro al usuario y no renderizar una lógica falsa.

Objetivo técnico inmediato: Implementar o mejorar el traductor:

Texto del usuario → JSON lógico simple → validación → compilación visual Ladder → render en herramienta de diagramas

No trabajar todavía la ejecución real en PLC salvo dejar preparado el JSON para enviarlo después al backend.

Entregables esperados: 1. Definir el contrato final del JSON lógico simple. 2. Crear o ajustar el endpoint/módulo que genera ese JSON desde lenguaje natural. 3. Crear un validador del JSON lógico. 4. Crear un compilador visual: logic_json → ladder_visual_schema 5. Integrar el resultado con la herramienta de diagramas Ladder. 6. Agregar ejemplos de prueba:

- directo
- OR
- AND
- NOT / paro general
- enclavamiento
- TON
- blinker
- Mantener separada la futura ejecución en PLC: logic_json → backend → registros PLC

Resumen de la decisión arquitectónica: La IA no programa el PLC. La IA no dibuja Ladder. La IA no genera tabla de verdad. La IA interpreta intención y genera JSON lógico simple. El frontend convierte ese JSON en visualización Ladder. El backend, en una etapa posterior, convertirá ese mismo JSON en cambios de registros para el PLC maestro.